PEP 585: Type Hinting Generics In Standard Collections

Status: Final · Created: 03-Mar-2019 · Public domain / CC0

Abstract

Static typing as defined by PEPs 484, 526, 544, 560, and 563 was built incrementally on top of the existing Python runtime and constrained by existing syntax and runtime behavior. This led to the existence of a duplicated collection hierarchy in the typing module due to generics (for example typing.List and the built-in list).

This PEP proposes to enable support for the generics syntax in all standard collections currently available in the typing module.

Rationale and Goals

This change removes the necessity for a parallel type hierarchy in the typing module, making it easier for users to annotate their programs and easier for teachers to teach Python.

Terminology

Generic (n.) -- a type that can be parameterized, typically a container. Also known as a *parametric type* or a *generic type*. For example: dict.

parameterized generic -- a specific instance of a generic with the expected types for container elements provided. Also known as a *parameterized type*. For example: dict[str, int].

Backwards compatibility

Tooling, including type checkers and linters, will have to be adapted to recognize standard collections as generics.

On the source level, the newly described functionality requires Python 3.9. For use cases restricted to type annotations, Python files with the "annotations" future-import (available since Python 3.7) can parameterize standard collections, including builtins. To reiterate, that depends on the external tools understanding that this is valid.

Implementation

Starting with Python 3.7, when from __future__ import annotations is used, function and variable annotations can parameterize standard collections directly. Example::

```
from __future__ import annotations
def find(haystack: dict[str, list[int]]) -> int:
    ...
```

Usefulness of this syntax before 585 is limited as external tooling like Mypy does not recognize standard collections as generic. Moreover, certain features of typing like type aliases or casting require putting types outside of annotations, in runtime context. While these are relatively less common than type annotations, it's important to allow using the same type syntax in all contexts. This is why starting with Python 3.9, the following collections become generic using __class_getitem__() to parameterize contained types:

* tuple # typing.Tuple * list # typing.List * dict # typing.Dict * set # typing.Set * frozenset # typing.FrozenSet * type # typing.Type * collections.deque * collections.defaultdict * collections.OrderedDict * collections.Counter * collections.ChainMap * collections.abc.Awaitable * collections.abc.Coroutine * collections.abc.AsyncIterable * collections.abc.AsyncIterator * collections.abc.AsyncGenerator * collections.abc.Iterable * collections.abc.Iterator *

collections.abc.Generator * collections.abc.Reversible * collections.abc.Container * collections.abc.Collection * collections.abc.Callable * collections.abc.Set # typing.AbstractSet * collections.abc.MutableSet * collections.abc.Mapping * collections.abc.MutableMapping * collections.abc.Sequence * collections.abc.MutableSequence * collections.abc.ByteString * collections.abc.MappingView * collections.abc.KeysView * collections.abc.ItemsView * collections.abc.ValuesView * contextlib.AbstractContextManager # typing.ContextManager * contextlib.AbstractAsyncContextManager # typing.AsyncContextManager * re.Pattern # typing.Pattern, typing.re.Pattern * re.Match # typing.Match, typing.re.Match

Importing those from typing is deprecated. Due to 563 and the intention to minimize the runtime impact of typing, this deprecation will not generate DeprecationWarnings. Instead, type checkers may warn about such deprecated usage when the target version of the checked program is signalled to be Python 3.9 or newer. It's recommended to allow for those warnings to be silenced on a project-wide basis.

The deprecated functionality may eventually be removed from the typing module. Removal will occur no sooner than Python 3.9's end of life, scheduled for October 2025.

Parameters to generics are available at runtime

Preserving the generic type at runtime enables introspection of the type which can be used for API generation or runtime type checking. Such usage is already present in the wild.

Just like with the typing module today, the parameterized generic types listed in the previous section all preserve their type parameters at runtime::

```
>>> list[str]
list[str]
>>> tuple[int, ...]
tuple[int, ...]
>>> ChainMap[str, list[str]]
collections.ChainMap[str, list[str]]
```

This is implemented using a thin proxy type that forwards all method calls and attribute accesses to the bare origin type with the following exceptions:

* the __repr__ shows the parameterized type; * the __origin__ attribute points at the non-parameterized generic class; * the __args__ attribute is a tuple (possibly of length 1) of generic types passed to the original __class_getitem__; * the __parameters__ attribute is a lazily computed tuple (possibly empty) of unique type variables found in __args__; * the __getitem__ raises an exception to disallow mistakes like dict[str][str]. However it allows e.g. dict[str, T][int] and in that case returns dict[str, int].

This design means that it is possible to create instances of parameterized collections, like::

```
>>> l = list[str]()
[]
>>> list is list[str]
False
>>> list == list[str]
False
>>> list[str] == list[str]
True
>>> list[str] == list[int]
False
>>> isinstance([1, 2, 3], list[str])
TypeError: isinstance() arg 2 cannot be a parameterized generic
>>> issubclass(list, list[str])
TypeError: issubclass() arg 2 cannot be a parameterized generic
>>> isinstance(list[str], types.GenericAlias)
True
```

Objects created with bare types and parameterized types are exactly the same. The generic parameters are not preserved in instances created with parameterized types, in other words generic types erase type parameters during object creation.

One important consequence of this is that the interpreter does **not** attempt to type check operations on the collection created with a parameterized type. This provides symmetry between::

```
l: list[str] = []
```

and::

```
l = list[str]()
```

For accessing the proxy type from Python code, it will be exported from the types module as GenericAlias.

Pickling or (shallow- or deep-) copying a GenericAlias instance will preserve the type, origin, attributes and parameters.

Forward compatibility

Future standard collections must implement the same behavior.

Reference implementation

A proof-of-concept or prototype implementation exists.

Rejected alternatives

Do nothing

Keeping the status quo forces Python programmers to perform book-keeping of imports from the typing module for standard collections, making all but the simplest annotations cumbersome to maintain. The existence of parallel types is confusing to newcomers (why is there both list and List?).

The above problems also don't exist in user-built generic classes which share runtime functionality and the ability to use them as generic type annotations. Making standard collections harder to use in type hinting from user classes hindered typing adoption and usability.

Generics erasure

It would be easier to implement __class_getitem__ on the listed standard collections in a way that doesn't preserve the generic type, in other words::

```
>>> list[str]
<class 'list'>
>>> tuple[int, ...]
<class 'tuple'>
>>> collections.ChainMap[str, list[str]]
<class 'collections.ChainMap'>
```

This is problematic as it breaks backwards compatibility: current equivalents of those types in the typing module **do** preserve the generic type::

```
>>> from typing import List, Tuple, ChainMap
>>> List[str]
typing.List[str]
>>> Tuple[int, ...]
typing.Tuple[int, ...]
>>> ChainMap[str, List[str]]
typing.ChainMap[str, typing.List[str]]
```

As mentioned in the "Implementation" section, preserving the generic type at runtime enables runtime introspection of the type which can be used for API generation or runtime type checking. Such usage is already present in the wild.

Additionally, implementing subscripts as identity functions would make Python less friendly to beginners. Say, if a user is mistakenly passing a list type instead of a list object to a function, and that function is indexing the received object, the code would no longer raise an error.

Today::

```
>>> l = list
```

```
>>> l[-1]
TypeError: 'type' object is not subscriptable
```

With __class_getitem__ as an identity function::

```
>>> l = list
>>> l[-1]
list
```

The indexing being successful here would likely end up raising an exception at a distance, confusing the user.

Disallowing instantiation of parameterized types

Given that the proxy type which preserves __origin__ and __args__ is mostly useful for runtime introspection purposes, we might have disallowed instantiation of parameterized types.

In fact, forbidding instantiation of parameterized types is what the typing module does today for types which parallel builtin collections (instantiation of other parameterized types is allowed).

The original reason for this decision was to discourage spurious parameterization which made object creation up to two orders of magnitude slower compared to the special syntax available for those builtin collections.

This rationale is not strong enough to allow the exceptional treatment of builtins. All other parameterized types can be instantiated, including parallels of collections in the standard library. Moreover, Python allows for instantiation of lists using list() and some builtin collections don't provide special syntax for instantiation.

Making isinstance(obj, list[str]) perform a check ignoring generics

An earlier version of this PEP suggested treating parameterized generics like list[str] as equivalent to their non-parameterized variants like list for purposes of isinstance() and issubclass(). This would be symmetrical to how list[str]() creates a regular list.

This design was rejected because isinstance() and issubclass() checks with parameterized generics would read like element-by-element runtime type checks. The result of those checks would be surprising, for example::

```
>>> isinstance([1, 2, 3], list[str])
True
```

Note the object doesn't match the provided generic type but isinstance() still returns True because it only checks whether the object is a list.

If a library is faced with a parameterized generic and would like to perform an isinstance() check using the base type, that type can be retrieved using the __origin__ attribute on the parameterized generic.

Making isinstance(obj, list[str]) perform a runtime type check

This functionality requires iterating over the collection which is a destructive operation in some of them. This functionality would have been useful, however implementing the type checker within Python that would deal with complex types, nested type checking, type variables, string forward references, and so on is out of scope for this PEP.

Naming the type GenericType instead of GenericAlias

We considered a different name for this type, but decided GenericAlias is better -- these aren't real types, they are aliases for the corresponding container type with some extra metadata attached.

Note on the initial draft

An early version of this PEP discussed matters beyond generics in standard collections. Those unrelated topics were removed for clarity.

Acknowledgments

Thank you to Guido van Rossum for his work on Python, and the implementation of this PEP specifically.

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.